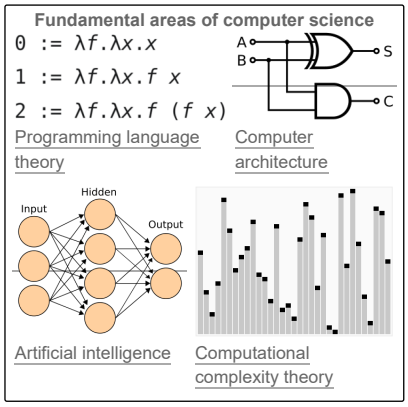
Computer science



Computer science is the study of computation, information, and automation.[1][2][3] Included broadly in the sciences, computer science spans theoretical disciplines (such as algorithms, theory of computation, and information theory) to applied disciplines (including the design and implementation of hardware and software).[4][5][6] An expert in the field is known as a computer scientist.

Algorithms and data structures are central to computer science.[7] The theory of computation concerns abstract models of computation and general classes of problems that can be solved using them. The fields of cryptography and computer security involve studying the means for secure communication and preventing security vulnerabilities. Computer graphics and computational geometry address the generation of images. Programming language theory considers different ways to describe computational processes, and database theory concerns the management of repositories of data. Human–computer interaction investigates the interfaces through which humans and computers interact, and software engineering focuses on the design and principles behind developing software. Areas such as operating systems, networks and embedded systems investigate the principles and design behind complex systems. Computer architecture describes the construction of computer components and computer-operated equipment. Artificial intelligence and machine learning aim to synthesize goal-orientated processes such as problem-solving, decision-making, environmental adaptation, planning and learning found in humans and animals. Within artificial intelligence, computer vision aims to understand and process image and video data, while natural language processing aims to understand and process textual and linguistic data.

The fundamental concern of computer science is determining what can and cannot be automated.[2][8][3][9][10] The Turing Award is generally recognized as the highest distinction in computer science.[11][12]

History

The earliest foundations of what would become computer science predate the invention of the modern digital computer. Machines for calculating fixed numerical tasks such as the abacus have existed since antiquity, aiding in computations such as multiplication and division. Algorithms for performing computations have existed since antiquity, even before the development of sophisticated computing equipment.[16]

Gottfried Wilhelm Leibniz (1646–1716) developed logic in a binary number system and has been called the "founder of computer science".[13]

Wilhelm Schickard designed and constructed the first working mechanical calculator in 1623.[17] In 1673, Gottfried Leibniz demonstrated a digital mechanical calculator, called the Stepped Reckoner.[18][19] Leibniz may be considered the first computer scientist and information theorist, because of various reasons, including the fact that he documented the binary number system. In 1820, Thomas de Colmar launched the mechanical calculator industry[note 1] when he invented his simplified arithmometer, the first calculating machine strong enough and reliable enough to be used daily in an office environment. Charles Babbage started the design of the first *automatic mechanical calculator*, his Difference Engine, in 1822, which eventually gave him the idea of the first *programmable mechanical calculator*, his Analytical Engine.[20] He started developing this machine in 1834, and "in less than two years, he had sketched out many of the salient features of the modern computer".[21] "A crucial step was the adoption of a punched card system derived from the Jacquard loom"[21] making it infinitely programmable.[note 2] In 1843, during the translation of a French article on the Analytical Engine, Ada Lovelace wrote, in one of the many notes she included, an algorithm to compute the Bernoulli numbers, which is considered to be the first published algorithm ever specifically tailored for implementation on a computer.[22] Around 1885, Herman Hollerith invented the tabulator, which used punched cards to process statistical information; eventually his company became part of IBM. Following Babbage, although unaware of his earlier work, Percy Ludgate in 1909 published[23] the 2nd of the only two designs for mechanical analytical engines in history. In 1914, the Spanish engineer Leonardo Torres Quevedo published his *Essays on Automatics*,[24] and designed, inspired by Babbage, a theoretical electromechanical calculating machine which was to be controlled by a read-only program. The paper also introduced the idea of floating-point arithmetic.[25][26] In 1920, to celebrate the 100th anniversary of the invention of the arithmometer, Torres presented in Paris the Electromechanical Arithmometer, a prototype that demonstrated the feasibility of an electromechanical analytical engine,[27] on which commands could be typed and the results printed automatically.[28] In 1937, one hundred years after Babbage's impossible dream, Howard Aiken convinced IBM, which was making all kinds of punched card equipment and was also in the calculator business[29] to develop his giant programmable calculator, the ASCC/Harvard Mark I, based on Babbage's Analytical Engine, which itself used punched cards and a central processing unit. When the machine was finished, some hailed it as "Babbage's dream come true".[30]

Charles Babbage is sometimes referred to as the "father of computing".[14]

During the 1940s, with the development of new and more powerful computing machines such as the Atanasoff–Berry computer and ENIAC, the term *computer* came to refer to the machines rather than their human predecessors.[31] As it became clear that computers could be used for more than just mathematical calculations, the field of computer science broadened to study computation in general. In 1945, IBM founded the Watson Scientific Computing Laboratory at Columbia University in New York City. The renovated fraternity house on Manhattan's West Side was IBM's first laboratory devoted to pure science. The lab is the forerunner of IBM's Research Division, which today operates research facilities around the world.[32] Ultimately, the close relationship between IBM and Columbia University was instrumental in the emergence of a new scientific discipline, with Columbia offering one of the first academic-credit courses in computer science in 1946.[33] Computer science began to be established as a distinct academic discipline in the 1950s and early 1960s.[34][35] The world's first computer science degree program, the Cambridge Diploma in Computer Science, began at the University of Cambridge Computer Laboratory in 1953. The first computer science department in the United States was formed at Purdue University in 1962.[36] Since practical computers became available, many applications of computing have become distinct areas of study in their own rights.

Etymology and scope

Ada Lovelace published the first algorithm intended for processing on a computer.[15]

Although first proposed in 1956,[37] the term "computer science" appears in a 1959 article in *Communications of the ACM*,[38] in which Louis Fein argues for the creation of a *Graduate School in Computer Sciences* analogous to the creation of Harvard Business School in 1921.[39] Louis justifies the name by arguing that, like management science, the subject is applied and interdisciplinary in nature, while having the characteristics typical of an academic discipline.[38] This effort, and those of others such as numerical analyst George Forsythe, were successful, and universities went on to create such departments, starting with Purdue in 1962.[40] Despite its name, a significant amount of computer science does not involve the study of computers themselves. Because of this, several alternative names have been proposed.[41] Certain departments of major universities prefer the term *computing science*, to emphasize precisely that difference. Danish scientist Peter Naur suggested the term *datalogy*,[42] to reflect the fact that the scientific discipline revolves around data and data treatment, while not necessarily involving computers. The first scientific institution to use the term was the Department of Datalogy at the University of Copenhagen, founded in 1969, with Peter Naur being the first professor in datalogy. The term is used mainly in the Scandinavian countries. An alternative term, also proposed by Naur, is data science; this is now used for a multi-disciplinary field of data analysis, including statistics and databases.

In the early days of computing, a number of terms for the practitioners of the field of computing were suggested (albeit facetiously) in the *Communications of the ACM*—*turingineer*, *turologist*, *flow-charts-man*, *applied meta-mathematician*, and *applied epistemologist*.[43] Three months later in the same journal, *comptologist* was suggested, followed next year by *hypologist*.[44] The term *computics* has also been suggested.[45] In Europe, terms derived from contracted translations of the expression "automatic information" (e.g. "*informazione automatica*" in Italian) or "information and mathematics" are often used, e.g. *informatique* (French), *Informatik* (German), *informatica* (Italian, Dutch), *informática* (Spanish, Portuguese), *informatika* (Slavic languages and Hungarian) or *pliroforiki* (πληροφορική, which means informatics) in Greek. Similar words have also been adopted in the UK (as in the School of Informatics, University of Edinburgh).[46] "In the U.S., however, informatics is linked with applied computing, or computing in the context of another domain."[47]

A folkloric quotation, often attributed to—but almost certainly not first formulated by—Edsger Dijkstra, states that "computer science is no more about computers than astronomy is about telescopes."[note 3] The design and deployment of computers and computer systems is generally considered the province of disciplines other than computer science. For example, the study of computer hardware is usually considered part of computer engineering, while the study of commercial computer systems and their deployment is often called information technology or information systems. However, there has been exchange of ideas between the various computer-related disciplines. Computer science research also often intersects other disciplines, such as cognitive science, linguistics, mathematics, physics, biology, Earth science, statistics, philosophy, and logic.

Computer science is considered by some to have a much closer relationship with mathematics than many scientific disciplines, with some observers saying that computing is a mathematical science.[34] Early computer science was strongly influenced by the work of mathematicians such as Kurt Gödel, Alan Turing, John von Neumann, Rózsa Péter, Stephen Kleene, and Alonzo Church and there continues to be a useful interchange of ideas between the two fields in areas such as mathematical logic, category theory, domain theory, and algebra.[37]

The relationship between computer science and software engineering is a contentious issue, which is further muddied by disputes over what the term "software engineering" means, and how computer science is defined.[48] David Parnas, taking a cue from the relationship between other engineering and science disciplines, has claimed that the principal focus of computer science is studying the properties of computation in general, while the principal focus of software engineering is the design of specific computations to achieve practical goals, making the two separate but complementary disciplines.[49]

The academic, political, and funding aspects of computer science tend to depend on whether a department is formed with a mathematical emphasis or with an engineering emphasis. Computer science departments with a mathematics emphasis and with a numerical orientation consider alignment with computational science. Both types of departments tend to make efforts to bridge the field educationally if not across all research.

Philosophy

Epistemology of computer science

Despite the word *science* in its name, there is debate over whether or not computer science is a discipline of science,[50] mathematics,[51] or engineering.[52] Allen Newell and Herbert A. Simon argued in 1975,

> Computer science is an empirical discipline. We would have called it an experimental science, but like astronomy, economics, and geology, some of its unique forms of observation and experience do not fit a narrow stereotype of the experimental method. Nonetheless, they are experiments. Each new machine that is built is an experiment. Actually constructing the machine poses a question to nature; and we listen for the answer by observing the machine in operation and analyzing it by all analytical and measurement means available.[52]

It has since been argued that computer science can be classified as an empirical science since it makes use of empirical testing to evaluate the correctness of programs, but a problem remains in defining the laws and theorems of computer science (if any exist) and defining the nature of experiments in computer science.[52] Proponents of classifying computer science as an engineering discipline argue that the reliability of computational systems is investigated in the same way as bridges in civil engineering and airplanes in aerospace engineering.[52] They also argue that while empirical sciences observe what presently exists, computer science observes what is possible to exist and while scientists discover laws from observation, no proper laws have been found in computer science and it is instead concerned with creating phenomena.[52]

Proponents of classifying computer science as a mathematical discipline argue that computer programs are physical realizations of mathematical entities and programs that can be deductively reasoned through mathematical formal methods.[52] Computer scientists Edsger W. Dijkstra and Tony Hoare regard instructions for computer programs as mathematical sentences and interpret formal semantics for programming languages as mathematical axiomatic systems.[52]

Paradigms of computer science

A number of computer scientists have argued for the distinction of three separate paradigms in computer science. Peter Wegner argued that those paradigms are science, technology, and mathematics.[53] Peter Denning's working group argued that they are theory, abstraction (modeling), and design.[34] Amnon H. Eden described them as the "rationalist paradigm" (which treats computer science as a branch of mathematics, which is prevalent in theoretical computer science, and mainly employs deductive reasoning), the "technocratic paradigm" (which might be found in engineering approaches, most prominently in software engineering), and the "scientific paradigm" (which approaches computer-related artifacts from the empirical perspective of natural sciences,[54] identifiable in some branches of artificial intelligence).[55] Computer science focuses on methods involved in design, specification, programming, verification, implementation and testing of human-made computing systems.[56]

Fields

As a discipline, computer science spans a range of topics from theoretical studies of algorithms and the limits of computation to the practical issues of implementing computing systems in hardware and software.[57][58] CSAB, formerly called Computing Sciences Accreditation Board—which is made up of representatives of the Association for Computing Machinery (ACM), and the IEEE Computer Society (IEEE CS)[59]—identifies four areas that it considers crucial to the discipline of computer science: *theory of computation*, *algorithms and data structures*, *programming methodology and languages*, and *computer elements and architecture*. In addition to these four areas, CSAB also identifies fields such as software engineering, artificial intelligence, computer networking and communication, database systems, parallel computation, distributed computation, human–computer interaction, computer graphics, operating systems, and numerical and symbolic computation as being important areas of computer science.[57]

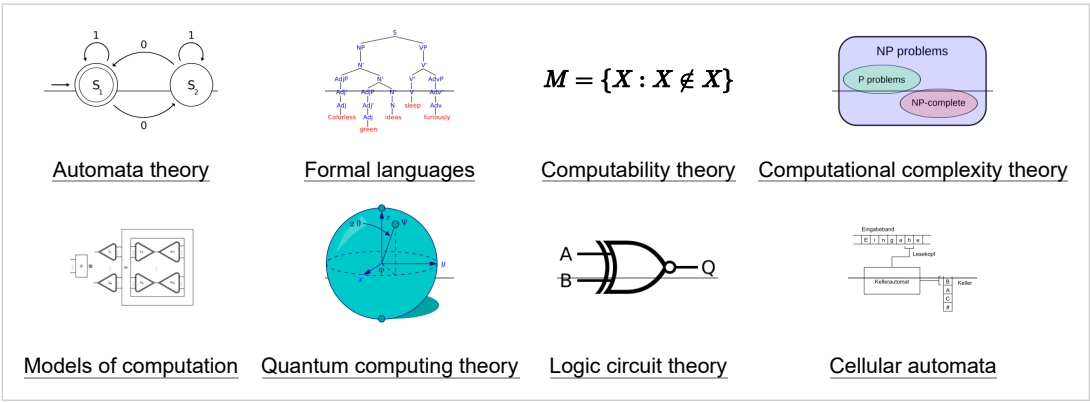
Theoretical computer science

Theoretical computer science is mathematical and abstract in spirit, but it derives its motivation from practical and everyday computation. It aims to understand the nature of computation and, as a consequence of this understanding, provide more efficient methodologies.

Theory of computation

According to Peter Denning, the fundamental question underlying computer science is, "What can be automated?"[3] Theory of computation is focused on answering fundamental questions about what can be computed and what amount of resources are required to perform those computations. In an effort to answer the first question, computability theory examines which computational problems are solvable on various theoretical models of computation. The second question is addressed by computational complexity theory, which studies the time and space costs associated with different approaches to solving a multitude of computational problems.

The famous P = NP? problem, one of the Millennium Prize Problems,[60] is an open problem in the theory of computation.

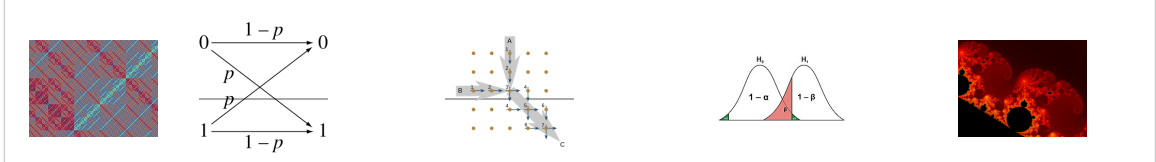


Automata theory | Formal languages | Computability theory | Computational complexity theory

Models of computation | Quantum computing theory | Logic circuit theory | Cellular automata

Information and coding theory

Information theory, closely related to probability and statistics, is related to the quantification of information. This was developed by Claude Shannon to find fundamental limits on signal processing operations such as compressing data and on reliably storing and communicating data.[61] Coding theory is the study of the properties of codes (systems for converting information from one form to another) and their fitness for a specific application. Codes are used for data compression, cryptography, error detection and correction, and more recently also for network coding. Codes are studied for the purpose of designing efficient and reliable data transmission methods.[62]



Coding theory | Channel capacity | Algorithmic information theory | Signal detection theory | Kolmogorov complexity

Data structures and algorithms

Data structures and algorithms are the studies of commonly used computational methods and their computational efficiency.

Analysis of algorithms | Algorithm design | Data structures | Combinatorial optimization | Computational geometry | Randomized algorithms

Programming language theory and formal methods

Programming language theory is a branch of computer science that deals with the design, implementation, analysis, characterization, and classification of programming languages and their individual features. It falls within the discipline of computer science, both depending on and affecting mathematics, software engineering, and linguistics. It is an active research area, with numerous dedicated academic journals.

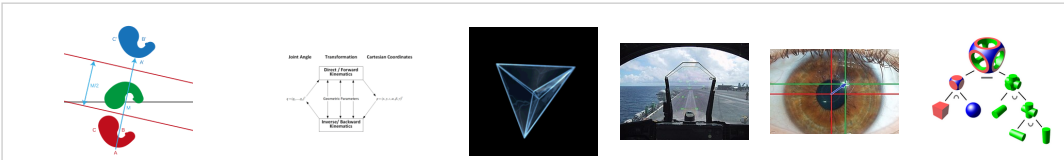
Formal methods are a particular kind of mathematically based technique for the specification, development and verification of software and hardware systems.[63] The use of formal methods for software and hardware design is motivated by the expectation that, as in other engineering disciplines, performing appropriate mathematical analysis can contribute to the reliability and robustness of a design. They form an important theoretical underpinning for software engineering, especially where safety or security is involved. Formal methods are a useful adjunct to software testing since they help avoid errors and can also give a framework for testing. For industrial use, tool support is required. However, the high cost of using formal methods means that they are usually only used in the development of high-integrity and life-critical systems, where safety or security is of utmost importance. Formal methods are best described as the application of a fairly broad variety of theoretical computer science fundamentals, in particular logic calculi, formal languages, automata theory, and program semantics, but also type systems and algebraic data types to problems in software and hardware specification and verification.

Formal semantics | Type theory | Compiler design | Programming languages | Formal verification | Automated theorem proving

Applied computer science

Computer graphics and visualization

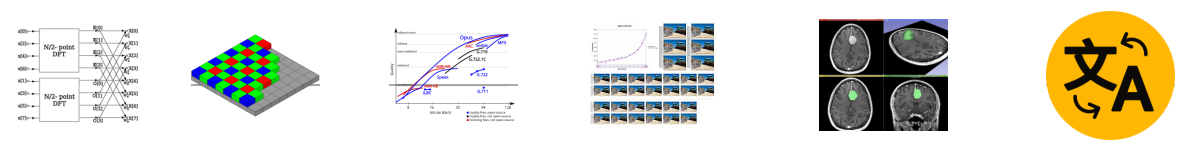
Computer graphics is the study of digital visual contents and involves the synthesis and manipulation of image data. The study is connected to many other fields in computer science, including computer vision, image processing, and computational geometry, and is heavily applied in the fields of special effects and video games.



2D computer graphics | Computer animation | Rendering | Mixed reality | Virtual reality | Solid modeling

Image and sound processing

Information can take the form of images, sound, video or other multimedia. Bits of information can be streamed via signals. Its processing is the central notion of informatics, the European view on computing, which studies information processing algorithms independently of the type of information carrier – whether it is electrical, mechanical or biological. This field plays important role in information theory, telecommunications, information engineering and has applications in medical image computing and speech synthesis, among others. *What is the lower bound on the complexity of fast Fourier transform algorithms?* is one of the unsolved problems in theoretical computer science.

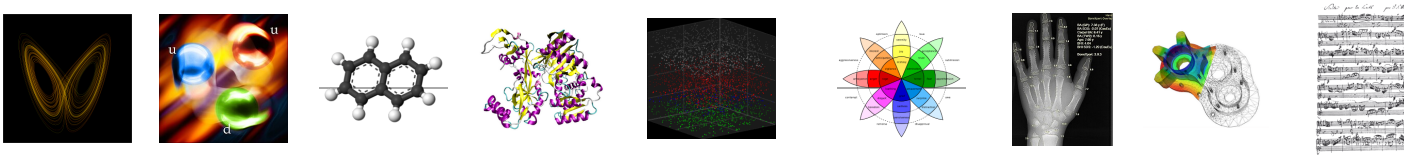


FFT algorithms | Image processing | Speech recognition | Data compression | Medical image computing | Speech synthesis

Computational science, finance and engineering

Scientific computing (or computational science) is the field of study concerned with constructing mathematical models and quantitative analysis techniques and using computers to analyze and solve scientific problems. A major usage of scientific computing is simulation of various processes, including computational fluid dynamics, physical, electrical, and electronic systems and circuits, societies and social situations (notably war games)

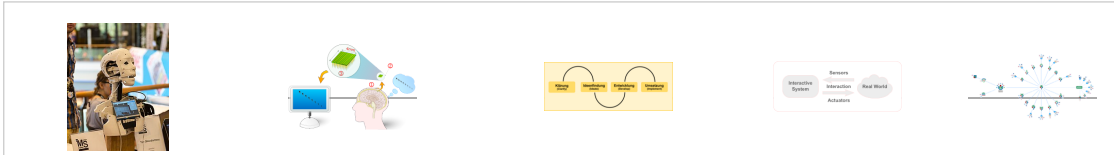
along with their habitats, and interactions among biological cells. Modern computers enable optimization of such designs as complete aircraft. Notable in electrical and electronic circuit design are SPICE,[64] as well as software for physical realization of new (or modified) designs. The latter includes essential design software for integrated circuits.[65]



Numerical analysis | Computational physics | Computational chemistry | Bioinformatics | Neuroinformatics | Psychoinformatics | Medical informatics | Computational engineering | Computatio musicolog

Human–computer interaction

Human–computer interaction (HCI) is the field of study and research concerned with the design and use of computer systems, mainly based on the analysis of the interaction between humans and computer interfaces. HCI has several subfields that focus on the relationship between emotions, social behavior and brain activity with computers.



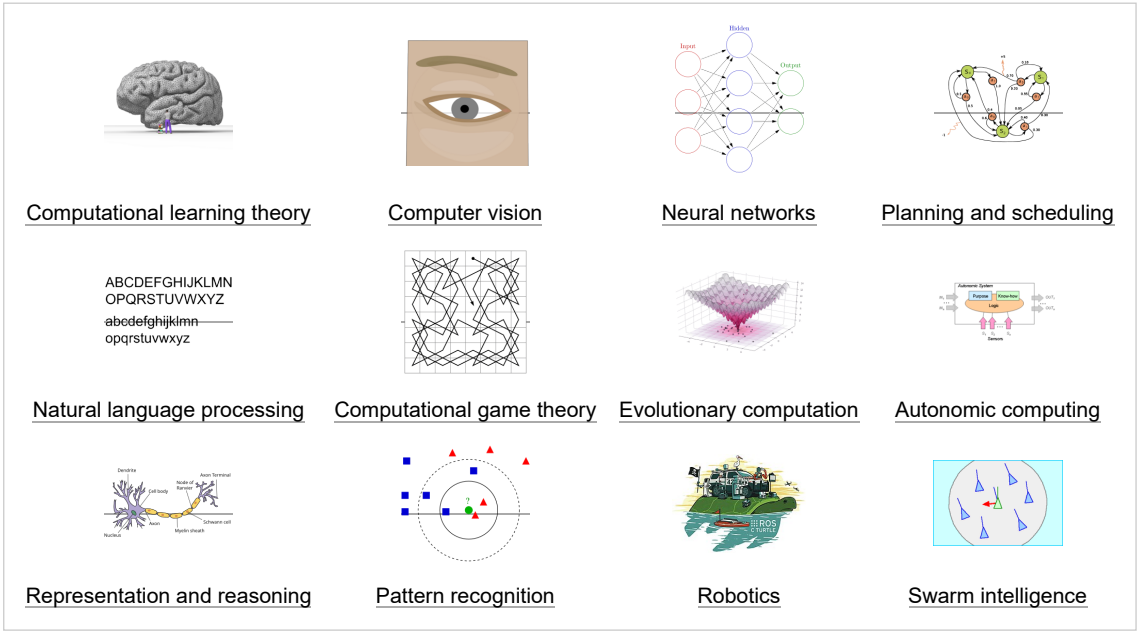
Affective computing | Brain–computer interface | Human-centered design | Physical computing | Social computing

Software engineering

Software engineering is the study of designing, implementing, and modifying the software in order to ensure it is of high quality, affordable, maintainable, and fast to build. It is a systematic approach to software design, involving the application of engineering practices to software. Software engineering deals with the organizing and analyzing of software—it does not just deal with the creation or manufacture of new software, but its internal arrangement and maintenance. For example software testing, systems engineering, technical debt and software development processes.

Artificial intelligence

Artificial intelligence (AI) aims to or is required to synthesize goal-orientated processes such as problem-solving, decision-making, environmental adaptation, learning, and communication found in humans and animals. From its origins in cybernetics and in the Dartmouth Conference (1956), artificial intelligence research has been necessarily cross-disciplinary, drawing on areas of expertise such as applied mathematics, symbolic logic, semiotics, electrical engineering, philosophy of mind, neurophysiology, and social intelligence. AI is associated in the popular mind with robotic development, but the main field of practical application has been as an embedded component in areas of software development, which require computational understanding. The starting point in the late 1940s was Alan Turing's question "Can computers think?", and the question remains effectively unanswered, although the Turing test is still used to assess computer output on the scale of human intelligence. But the automation of evaluative and predictive tasks has been increasingly successful as a substitute for human monitoring and intervention in domains of computer application involving complex real-world data.



Computational learning theory | Computer vision | Neural networks | Planning and scheduling

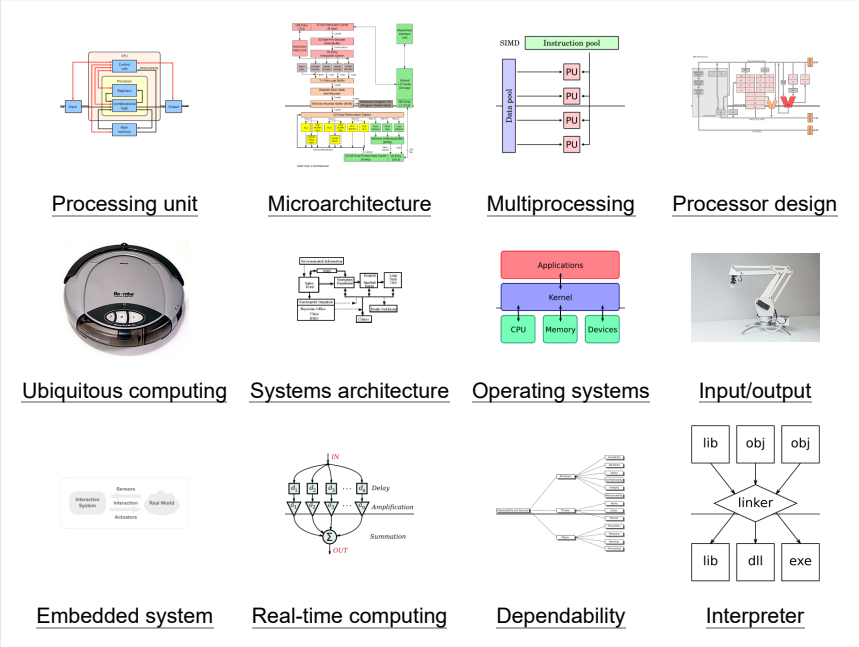
Natural language processing | Computational game theory | Evolutionary computation | Autonomic computing

Representation and reasoning | Pattern recognition | Robotics | Swarm intelligence

Computer systems

Computer architecture and microarchitecture

Computer architecture, or digital computer organization, is the conceptual design and fundamental operational structure of a computer system. It focuses largely on the way by which the central processing unit performs internally and accesses addresses in memory.[66] Computer engineers study computational logic and design of computer hardware, from individual processor components, microcontrollers, personal computers to supercomputers and embedded systems. The term "architecture" in computer literature can be traced to the work of Lyle R. Johnson and Frederick P. Brooks Jr., members of the Machine Organization department in IBM's main research center in 1959.



Concurrent, parallel and distributed computing

Concurrency is a property of systems in which several computations are executing simultaneously, and potentially interacting with each other.[67] A number of mathematical models have been developed for general concurrent computation including Petri nets, process calculi and the parallel random access machine model.[68] When multiple computers are connected in a network while using concurrency, this is known as a distributed system. Computers within that distributed system have their own private memory, and information can be exchanged to achieve common goals.[69]

Computer networks

This branch of computer science aims studies the construction and behavior of computer networks. It addresses their performance, resilience, security, scalability, and cost-effectiveness, along with the variety of services they can provide.[70]

Computer security and cryptography

Computer security is a branch of computer technology with the objective of protecting information from unauthorized access, disruption, or modification while maintaining the accessibility and usability of the system for its intended users.

Historical cryptography is the art of writing and deciphering secret messages. Modern cryptography is the scientific study of problems relating to distributed computations that can be attacked.[71] Technologies studied in modern cryptography include symmetric and asymmetric encryption, digital signatures, cryptographic hash functions, key-agreement protocols, blockchain, zero-knowledge proofs, and garbled circuits.

Databases and data mining

A database is intended to organize, store, and retrieve large amounts of data easily. Digital databases are managed using database management systems to store, create, maintain, and search data, through database models and query languages. Data mining is a process of discovering patterns in large data sets.

Discoveries

The philosopher of computing Bill Rapaport noted three *Great Insights of Computer Science*:[72]

- Gottfried Wilhelm Leibniz's, George Boole's, Alan Turing's, Claude Shannon's, and Samuel Morse's insight: there are only *two objects* that a computer has to deal with in order to represent "anything".[note 4]

  > All the information about any computable problem can be represented using only 0 and 1 (or any other bistable pair that can flip-flop between two easily distinguishable states, such as "on/off", "magnetized/de-magnetized", "high-voltage/low-voltage", etc.).

- Alan Turing's insight: there are only *five actions* that a computer has to perform in order to do "anything".

  > Every algorithm can be expressed in a language for a computer consisting of only five basic instructions:[73]
  >
  > - move left one location;

- move right one location;
- read symbol at current location;
- print 0 at current location;
- print 1 at current location.

- Corrado Böhm and Giuseppe Jacopini's insight: there are only *three ways of combining* these actions (into more complex ones) that are needed in order for a computer to do "anything".[74]

  Only three rules are needed to combine any set of basic instructions into more complex ones:

  - *sequence*: first do this, then do that;
  - *selection*: IF such-and-such is the case, THEN do this, ELSE do that;
  - *repetition*: WHILE such-and-such is the case, DO this.

  The three rules of Boehm's and Jacopini's insight can be further simplified with the use of goto (which means it is more elementary than structured programming).

Programming paradigms

Programming languages can be used to accomplish different tasks in different ways. Common programming paradigms include:

- Functional programming, a style of building the structure and elements of computer programs that treats computation as the evaluation of mathematical functions and avoids state and mutable data. It is a declarative programming paradigm, which means programming is done with expressions or declarations instead of statements.[75]
- Imperative programming, a programming paradigm that uses statements that change a program's state.[76] In much the same way that the imperative mood in natural languages expresses commands, an imperative program consists of commands for the computer to perform. Imperative programming focuses on describing how a program operates.
- Object-oriented programming, a programming paradigm based on the concept of "objects", which may contain data, in the form of fields, often known as attributes; and code, in the form of procedures, often known as methods. A feature of objects is that an object's procedures can access and often modify the data fields of the object with which they are associated. Thus object-oriented computer programs are made out of objects that interact with one another.[77]
- Service-oriented programming, a programming paradigm that uses "services" as the unit of computer work, to design and implement integrated business applications and mission critical software programs.

Many languages offer support for multiple paradigms, making the distinction more a matter of style than of technical capabilities.[78]

Research

Conferences are important events for computer science research. During these conferences, researchers from the public and private sectors present their recent work and meet. Unlike in most other academic fields, in computer science, the prestige of conference papers is greater than that of journal publications.[79][80] One proposed explanation for this is the quick development of this relatively new field requires rapid review and distribution of results, a task better handled by conferences than by journals.[81]

See also

- Computer science education
- Glossary of computer science
- List of computer scientists
- List of computer science awards
- Electronics and Computer Engineering
- List of pioneers in computer science
- Outline of computer science

Notes

1. In 1851
2. "The introduction of punched cards into the new engine was important not only as a more convenient form of control than the drums, or because programs could now be of unlimited extent, and could be stored and repeated without the danger of introducing errors in setting the machine by hand; it was important also because it served to crystallize Babbage's feeling that he had invented something really new, something much more than a sophisticated calculating machine." Bruce Collier, 1970
3. See the entry "Computer science" on Wikiquote for the history of this quotation.
4. The word "anything" is written in quotation marks because there are things that computers cannot do. One example is: to answer the question if an arbitrary given computer program will eventually finish or run forever (the Halting problem).

References

1. "What is Computer Science?" (https://www.cs.york.ac.uk/undergraduate/what-is-cs/). *Department of Computer Science, University of York*. Archived (https://web.archive.org/web/20200611230638/https://www.cs.york.ac.uk/undergraduate/what-is-cs/) from the original on June 11, 2020. Retrieved June 11, 2020.
2. *What Can Be Automated? Computer Science and Engineering Research Study* (https://mitpress.mit.edu/books/what-can-be-automated). Computer Science Series. MIT Press. 1980. ISBN 978-0262010603. Archived (https://web.archive.org/web/20210109021022/https://mitpress.mit.edu/books/what-can-be-automated) from the original on January 9, 2021.
3. Denning, P.J.; Comer, D.E.; Gries, D.; Mulder, M.C.; Tucker, A.; Turner, A.J.; Young, P.R. (February 1989). "Computing as a discipline". *Computer*. **22** (2): 63–70. Bibcode:1989Compr..22b..63D (https://ui.adsabs.harvard.edu/abs/1989Compr..22b..63D). doi:10.1109/2.19833 (https://doi.org/10.1109%2F2.19833). ISSN 1558-0814 (https://search.worldcat.org/issn/1558-0814). "The discipline of computing is the systematic study of algorithmic processes that describe and transform information, their theory, analysis, design, efficiency, implementation, and application. The fundamental question underlying all of computing is, 'What can be (efficiently) automated?' "

4. "WordNet Search—3.1" (http://wordnetweb.princeton.edu/perl/webwn?s=computer%20scientist). *WordNet Search*. Wordnetweb.princeton.edu. Archived (https://web.archive.org/web/20171018181122/http://wordnetweb.princeton.edu/perl/webwn?s=computer%20scientist) from the original on October 18, 2017. Retrieved May 14, 2012.
5. "Definition of computer science | Dictionary.com" (https://www.dictionary.com/browse/computer-science). *www.dictionary.com*. Archived (https://web.archive.org/web/20200611224238/https://www.dictionary.com/browse/computer-science) from the original on June 11, 2020. Retrieved June 11, 2020.
6. "What is Computer Science? | Undergraduate Computer Science at UMD" (https://undergrad.cs.umd.edu/what-computer-science). *undergrad.cs.umd.edu*. Archived (https://web.archive.org/web/20201127013803/https://undergrad.cs.umd.edu/what-computer-science) from the original on November 27, 2020. Retrieved July 15, 2022.
7. Harel, David (2014). *Algorithmics The Spirit of Computing*. Springer Berlin. ISBN 978-3-642-44135-6. OCLC 876384882 (https://search.worldcat.org/oclc/876384882).
8. Patton, Richard D.; Patton, Peter C. (2009), Nof, Shimon Y. (ed.), "What Can be Automated? What Cannot be Automated?" (https://doi.org/10.1007/978-3-540-78831-7_18), *Springer Handbook of Automation*, Springer Handbooks, Berlin, Heidelberg: Springer, pp. 305–313, doi:10.1007/978-3-540-78831-7_18 (https://doi.org/10.1007%2F978-3-540-78831-7_18), ISBN 978-3-540-78831-7, archived (https://web.archive.org/web/20230111224039/https://link.springer.com/chapter/10.1007/978-3-540-78831-7_18) from the original on January 11, 2023, retrieved March 3, 2022
9. Forsythe, George (August 5–10, 1969). "Computer Science and Education". *Proceedings of IFIP Congress 1968*. "The question 'What can be automated?' is one of the most inspiring philosophical and practical questions of contemporary civilization."
10. Knuth, Donald E. (August 1, 1972). "George Forsythe and the development of computer science" (https://doi.org/10.1145%2F361532.361538). *Communications of the ACM*. **15** (8): 721–726. doi:10.1145/361532.361538 (https://doi.org/10.1145%2F361532.361538). ISSN 0001-0782 (https://search.worldcat.org/issn/0001-0782). S2CID 12512057 (https://api.semanticscholar.org/CorpusID:12512057).
11. Hanson, Vicki L. (January 23, 2017). "Celebrating 50 years of the Turing award" (https://doi.org/10.1145%2F3033604). *Communications of the ACM*. **60** (2): 5. doi:10.1145/3033604 (https://doi.org/10.1145%2F3033604). ISSN 0001-0782 (https://search.worldcat.org/issn/0001-0782). S2CID 29984960 (https://api.semanticscholar.org/CorpusID:29984960).
12. Scott, Eric; Martins, Marcella Scoczynski Ribeiro; Yafrani, Mohamed El; Volz, Vanessa; Wilson, Dennis G (June 5, 2018). "ACM marks 50 years of the ACM A.M. turing award and computing's greatest achievements" (https://doi.org/10.1145/3231560.3231563). *ACM SIGEVOlution*. **10** (3): 9–11. doi:10.1145/3231560.3231563 (https://doi.org/10.1145%2F3231560.3231563). ISSN 1931-8499 (https://search.worldcat.org/issn/1931-8499). S2CID 47021559 (https://api.semanticscholar.org/CorpusID:47021559).
13. "2021: 375th birthday of Leibniz, father of computer science" (https://people.idsia.ch/~juergen/leibniz-father-computer-science-375.html). *people.idsia.ch*. Archived (https://web.archive.org/web/20220921232935/https://people.idsia.ch/~juergen/leibniz-father-computer-science-375.html) from the original on September 21, 2022. Retrieved February 4, 2023.
14. "Charles Babbage Institute: Who Was Charles Babbage?" (http://www.cbi.umn.edu/about/babbage.html). *cbi.umn.edu*. Archived (https://web.archive.org/web/20070109093346/http://www.cbi.umn.edu/about/babbage.html) from the original on January 9, 2007. Retrieved December 28, 2016.
15. "Ada Lovelace | Babbage Engine | Computer History Museum" (http://www.computerhistory.org/babbage/adalovelace/). *www.computerhistory.org*. Archived (https://web.archive.org/web/20181225024329/http://www.computerhistory.org/babbage/adalovelace/%20) from the original on December 25, 2018. Retrieved December 28, 2016.
16. "History of Computer Science" (https://cs.uwaterloo.ca/~shallit/Courses/134/history.html#:~:text=In%20the%201960%27s,%20computer%20science,person%20to%20receive%20a%20Ph.). *cs.uwaterloo.ca*. Archived (https://web.archive.org/web/20170729210116/https://cs.uwaterloo.ca/~shallit/Courses/134/history.html#:~:text=In%20the%201960%27s,%20computer%20science,person%20to%20receive%20a%20Ph.) from the original on July 29, 2017. Retrieved July 15, 2022.
17. "Wilhelm Schickard – Ein Computerpionier" (https://web.archive.org/web/20200919014352/https://www.fmi.uni-jena.de/fmimedia/Fakultaet/Institute+und+Abteilungen/Abteilung+f%C3%BCr+Didaktik/GDI/Wilhelm+Schickard.pdf) (PDF) (in German). Archived from the original (http://www.fmi.uni-jena.de/fmimedia/Fakultaet/Institute+und+Abteilungen/Abteilung+f%C3%BCr+Didaktik/GDI/Wilhelm+Schickard.pdf) (PDF) on September 19, 2020. Retrieved December 4, 2016.
18. "Pre-History of Computing" (https://www.i-programmer.info/history/machines/517-the-prehistory-of-computers.html?start=1). *www.i-programmer.info*. Retrieved March 16, 2026.
19. "The Leibniz Step Reckoner and Curta Calculators" (https://www.computerhistory.org/revolution/story/49). *Computer History Museum*. Retrieved March 16, 2026.
20. "Science Museum, Babbage's Analytical Engine, 1834–1871 (Trial model)" (https://collection.sciencemuseumgroup.org.uk/objects/co62245/babbages-analytical-engine-1834-1871-trial-model-analytical-engines). Archived (https://web.archive.org/web/20190830123359/https://collection.sciencemuseumgroup.org.uk/objects/co62245/babbages-analytical-engine-1834-1871-trial-model-analytical-engines) from the original on August 30, 2019. Retrieved May 11, 2020.
21. Hyman, Anthony (1982). *Charles Babbage: Pioneer of the Computer*. Oxford University Press. ISBN 978-0691083032.
22. "A Selection and Adaptation From Ada's Notes found in Ada, The Enchantress of Numbers," by Betty Alexandra Toole Ed.D. Strawberry Press, Mill Valley, CA" (https://web.archive.org/web/20060210172109/http://www.scottlan.edu/lriddle/women/ada-love.htm). Archived from the original (http://www.scottlan.edu/Lriddle/women/ada-love.htm) on February 10, 2006. Retrieved May 4, 2006.
23. "The John Gabriel Byrne Computer Science Collection" (https://web.archive.org/web/20190416071721/https://www.scss.tcd.ie/SCSSTreasuresCatalog/miscellany/TCD-SCSS-X.20121208.002/TCD-SCSS-X.20121208.002.pdf) (PDF). Archived from the original (https://scss.tcd.ie/SCSSTreasuresCatalog/miscellany/TCD-SCSS-X.20121208.002/TCD-SCSS-X.20121208.002.pdf/) on April 16, 2019. Retrieved August 8, 2019.
24. Torres Quevedo, L. (1914). "Ensayos sobre Automática – Su definicion. Extension teórica de sus aplicaciones". *Revista de la Academia de Ciencias Exacta*, 12, pp. 391–418.
25. Torres Quevedo, Leonardo. Automática: Complemento de la Teoría de las Máquinas, (pdf) (https://quickclick.es/rop/pdf/publico/1914/1914_tomol_2043_01.pdf), pp. 575–583, Revista de Obras Públicas, 19 November 1914.
26. Ronald T. Kneusel. *Numbers and Computers (https://books.google.com/books?id=eq4ZDgAAQBAJ&dq=leonardo+torres+quevedo++electromechanical+machine+essays&pg=PA84),* Springer, pp. 84–85, 2017. ISBN 978-3319505084
27. Randell, Brian. Digital Computers, History of Origins, (pdf) (https://dl.acm.org/doi/pdf/10.5555/1074100.1074334), p. 545, Digital Computers: Origins, Encyclopedia of Computer Science, January 2003.
28. Randell 1982, p. 6, 11–13.
29. "In this sense Aiken needed IBM, whose technology included the use of punched cards, the accumulation of numerical data, and the transfer of numerical data from one register to another", Bernard Cohen, p.44 (2000)
30. Brian Randell, p. 187, 1975
31. The Association for Computing Machinery (ACM) was founded in 1947.
32. "IBM Archives: 1945" (https://www.ibm.com/ibm/history/history/year_1945.html). Ibm.com. January 23, 2003. Archived (https://web.archive.org/web/20190105013948/https://www.ibm.com/ibm/history/history/year_1945.html) from the original on January 5, 2019. Retrieved March 19, 2019.
33. "IBM100 – The Origins of Computer Science" (https://www.ibm.com/ibm/history/ibm100/us/en/icons/compsci/). Ibm.com. September 15, 1995. Archived (https://web.archive.org/web/20190105051800/https://www.ibm.com/ibm/history/ibm100/us/en/icons/compsci/) from the original on January 5, 2019. Retrieved March 19, 2019.
34. Denning, P.J.; Comer, D.E.; Gries, D.; Mulder, M.C.; Tucker, A.; Turner, A.J.; Young, P.R. (February 1989). "Computing as a discipline". *Computer*. **22** (2): 63–70. Bibcode:1989Compr..22b..63D (https://ui.adsabs.harvard.edu/abs/1989Compr..22b..63D). doi:10.1109/2.19833 (https://doi.org/10.1109%2F2.19833). ISSN 1558-0814 (https://search.worldcat.org/issn/1558-0814).
35. "Some EDSAC statistics" (http://www.cl.cam.ac.uk/conference/EDSAC99/statistics.html). University of Cambridge. Archived (https://web.archive.org/web/20070903055322/http://www.cl.cam.ac.uk/conference/EDSAC99/statistics.html) from the original on September 3, 2007. Retrieved November 19, 2011.
36. "Computer science pioneer Samuel D. Conte dies at 85" (http://www.cs.purdue.edu/about/conte.html). Purdue Computer Science. July 1, 2002. Archived (https://web.archive.org/web/20141006142241/http://www.cs.purdue.edu/about/conte.html) from the original on October 6, 2014. Retrieved December 12, 2014.
37. Tedre, Matti (2014). *The Science of Computing: Shaping a Discipline*. Taylor and Francis / CRC Press.

38. Louis Fine (1960). "The Role of the University in Computers, Data Processing, and Related Fields" (https://doi.org/10.1145%2F368424.368427). *Communications of the ACM*. **2** (9): 7–14. doi:10.1145/368424.368427 (https://doi.org/10.1145%2F368424.368427). S2CID 6740821 (https://api.semanticscholar.org/CorpusID:6740821).
39. "Stanford University Oral History" (http://library.stanford.edu/guides/stanford-university-oral-history). *Stanford Libraries*. Stanford University. Archived (https://web.archive.org/web/20170404070555/http://library.stanford.edu/guides/stanford-university-oral-history) from the original on April 4, 2017. Retrieved May 30, 2013.
40. Donald Knuth (1972). *"George Forsythe and the Development of Computer Science" (http://www.stanford.edu/dept/ICME/docs/history/forsythe_knuth.pdf)*. *Comms. ACM*. Archived (https://web.archive.org/web/20131020200802/http://www.stanford.edu/dept/ICME/docs/history/forsythe_knuth.pdf) October 20, 2013, at the Wayback Machine
41. Matti Tedre (2006). "The Development of Computer Science: A Sociocultural Perspective" (http://epublications.uef.fi/pub/urn_isbn_952-458-867-6/urn_isbn_952-458-867-6.pdf) (PDF). p. 260. Archived (https://ghostarchive.org/archive/20221009/http://epublications.uef.fi/pub/urn_isbn_952-458-867-6/urn_isbn_952-458-867-6.pdf) (PDF) from the original on October 9, 2022. Retrieved December 12, 2014.
42. Peter Naur (1966). "The science of datalogy" (https://doi.org/10.1145%2F365719.366510). *Communications of the ACM*. **9** (7): 485. doi:10.1145/365719.366510 (https://doi.org/10.1145%2F365719.366510). S2CID 47558402 (https://api.semanticscholar.org/CorpusID:47558402).
43. Weiss, E.A.; Corley, Henry P.T. "Letters to the editor" (https://doi.org/10.1145%2F368796.368802). *Communications of the ACM*. **1** (4): 6. doi:10.1145/368796.368802 (https://doi.org/10.1145%2F368796.368802). S2CID 5379449 (https://api.semanticscholar.org/CorpusID:5379449).
44. Communications of the ACM 2(1):p.4
45. IEEE Computer 28(12): p.136
46. P. Mounier-Kuhn, *L'Informatique en France, de la seconde guerre mondiale au Plan Calcul. L'émergence d'une science*, Paris, PUPS, 2010, ch. 3 & 4.
47. Groth, Dennis P. (February 2010). "Why an Informatics Degree?" (http://cacm.acm.org/magazines/2010/2/69363-why-an-informatics-degree). *Communications of the ACM*. Cacm.acm.org. Archived (https://web.archive.org/web/20230111224014/https://cacm.acm.org/magazines/2010/2/69363-why-an-informatics-degree/abstract) from the original on January 11, 2023. Retrieved June 14, 2016.
48. Tedre, M. (2011). "Computing as a Science: A Survey of Competing Viewpoints". *Minds and Machines*. **21** (3): 361–387. doi:10.1007/s11023-011-9240-4 (https://doi.org/10.1007%2Fs11023-011-9240-4). S2CID 14263916 (https://api.semanticscholar.org/CorpusID:14263916).
49. Parnas, D.L. (1998). "Software engineering programmes are not computer science programmes". *Annals of Software Engineering*. **6** (1–4): 19–37. doi:10.1023/A:1018949113292 (https://doi.org/10.1023%2FA%3A1018949113292). S2CID 35786237 (https://api.semanticscholar.org/CorpusID:35786237)., p. 19: "Rather than treat software engineering as a subfield of computer science, I treat it as an element of the set, Civil Engineering, Mechanical Engineering, Chemical Engineering, Electrical Engineering, [...]"
50. Luk, R.W.P. (2020). "Insight in how computer science can be a science". *Science & Philosophy*. **8** (2): 17–47. doi:10.23756/sp.v8i2.531 (https://doi.org/10.23756%2Fsp.v8i2.531).
51. Knuth, D.E. (1974). "Computer science and its relation to mathematics". *The American Mathematical Monthly*. **81** (4): 323–343. doi:10.2307/2318994 (https://doi.org/10.2307%2F2318994). JSTOR 2318994 (https://www.jstor.org/stable/2318994).
52. "The Philosophy of Computer Science" (https://plato.stanford.edu/entries/computer-science/#EpisStatCompScie). *The Philosophy of Computer Science (Stanford Encyclopedia of Philosophy)*. Metaphysics Research Lab, Stanford University. 2021. Archived (https://web.archive.org/web/20210916211931/https://plato.stanford.edu/entries/computer-science/#EpisStatCompScie) from the original on September 16, 2021. Retrieved September 16, 2021.
53. Wegner, P. (October 13–15, 1976). *Research paradigms in computer science—Proceedings of the 2nd international Conference on Software Engineering*. San Francisco, California, United States: IEEE Computer Society Press, Los Alamitos, CA.
54. Denning, Peter J. (2007). "Computing is a natural science". *Communications of the ACM*. **50** (7): 13–18. doi:10.1145/1272516.1272529 (https://doi.org/10.1145%2F1272516.1272529). S2CID 20045303 (https://api.semanticscholar.org/CorpusID:20045303).
55. Eden, A.H. (2007). "Three Paradigms of Computer Science" (https://web.archive.org/web/20160215100211/http://www.eden-study.org/articles/2007/three_paradigms_of_computer_science.pdf) (PDF). *Minds and Machines*. **17** (2): 135–167. CiteSeerX 10.1.1.304.7763 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.304.7763). doi:10.1007/s11023-007-9060-8 (https://doi.org/10.1007%2Fs11023-007-9060-8). S2CID 3023076 (https://api.semanticscholar.org/CorpusID:3023076). Archived from the original (http://www.eden-study.org/articles/2007/three_paradigms_of_computer_science.pdf) (PDF) on February 15, 2016.
56. Turner, Raymond; Angius, Nicola (2019). "The Philosophy of Computer Science" (https://plato.stanford.edu/archives/spr2019/entries/computer-science/). In Zalta, Edward N. (ed.). *The Stanford Encyclopedia of Philosophy*. Archived (https://web.archive.org/web/20191014101624/https://plato.stanford.edu/archives/spr2019/entries/computer-science/) from the original on October 14, 2019. Retrieved October 14, 2019.
57. "Computer Science as a Profession" (https://web.archive.org/web/20080617030847/http://www.csab.org/comp_sci_profession.html). Computing Sciences Accreditation Board. May 28, 1997. Archived from the original (http://www.csab.org/comp_sci_profession.html) on June 17, 2008. Retrieved May 23, 2010.
58. Committee on the Fundamentals of Computer Science: Challenges and Opportunities, National Research Council (2004). *Computer Science: Reflections on the Field, Reflections from the Field* (http://www.nap.edu/catalog.php?record_id=11106#toc). National Academies Press. ISBN 978-0-309-09301-9. Archived (https://web.archive.org/web/20110218122042/http://www.nap.edu/catalog.php?record_id=11106#toc) from the original on February 18, 2011. Retrieved August 31, 2008.
59. "CSAB Leading Computer Education" (http://www.csab.org/). CSAB. August 3, 2011. Archived (https://web.archive.org/web/20190120190336/http://www.csab.org/) from the original on January 20, 2019. Retrieved November 19, 2011.
60. Clay Mathematics Institute (http://www.claymath.org/millennium/P_vs_NP/) P = NP Archived (https://web.archive.org/web/20131014194456/http://www.claymath.org/millennium/P_vs_NP/) October 14, 2013, at the Wayback Machine
61. P. Collins, Graham (October 14, 2002). "Claude E. Shannon: Founder of Information Theory" (http://www.scientificamerican.com/article.cfm?id=claude-e-shannon-founder). *Scientific American*. Archived (https://web.archive.org/web/20140116183558/http://www.scientificamerican.com/article.cfm?id=claude-e-shannon-founder) from the original on January 16, 2014. Retrieved December 12, 2014.
62. Van-Nam Huynh; Vladik Kreinovich; Songsak Sriboonchitta; 2012. *Uncertainty Analysis in Econometrics with Applications*. Springer Science & Business Media. p. 63. ISBN 978-3-642-35443-4.
63. Phillip A. Laplante, (2010). *Encyclopedia of Software Engineering* Three-Volume Set (Print). CRC Press. p. 309. ISBN 978-1-351-24926-3.
64. Muhammad H. Rashid, (2016). *SPICE for Power Electronics and Electric Power*. CRC Press. p. 6. ISBN 978-1-4398-6047-2.
65. "What is an integrated circuit (IC)? A vital component of modern electronics" (https://whatis.techtarget.com/definition/integrated-circuit-IC). *WhatIs.com*. Archived (https://web.archive.org/web/20211115153823/https://whatis.techtarget.com/definition/integrated-circuit-IC) from the original on November 15, 2021. Retrieved November 15, 2021.
66. A. Thisted, Ronald (April 7, 1997). "Computer Architecture" (http://galton.uchicago.edu/~thisted/Distribute/comparch.pdf) (PDF). The University of Chicago. Archived (https://ghostarchive.org/archive/20221009/http://galton.uchicago.edu/~thisted/Distribute/comparch.pdf) (PDF) from the original on October 9, 2022.
67. Jiacun Wang, (2017). *Real-Time Embedded Systems*. Wiley. p. 12. ISBN 978-1-119-42070-5.
68. Gordana Dodig-Crnkovic; Raffaela Giovagnoli, (2013). *Computing Nature: Turing Centenary Perspective*. Springer Science & Business Media. p. 247. ISBN 978-3-642-37225-4.
69. Simon Elias Bibri (2018). *Smart Sustainable Cities of the Future: The Untapped Potential of Big Data Analytics and Context-Aware Computing for Advancing Sustainability*. Springer. p. 74. ISBN 978-3-319-73981-6.
70. Peterson, Larry; Davie, Bruce (2000). *Computer Networks: A Systems Approach* (https://book.systemsapproach.org/index.html). Singapore: Harcourt Asia. ISBN 9789814066433. Retrieved May 24, 2025.
71. Katz, Jonathan (2008). *Introduction to modern cryptography*. Yehuda Lindell. Boca Raton: Chapman & Hall/CRC. ISBN 978-1-58488-551-1. OCLC 137325053 (https://search.worldcat.org/oclc/137325053).
72. Rapaport, William J. (September 20, 2013). "What Is Computation?" (http://www.cse.buffalo.edu/~rapaport/computation.html). State University of New York at Buffalo. Archived (https://web.archive.org/web/20010214002845/http://www.cse.buffalo.edu/~rapaport/computation.html) from the original on February 14, 2001. Retrieved August 31, 2013.

73. B. Jack Copeland, (2012). *Alan Turing's Electronic Brain: The Struggle to Build the ACE, the World's Fastest Computer*. OUP Oxford. p. 107. ISBN 978-0-19-960915-4.
74. Charles W. Herbert, (2010). *An Introduction to Programming Using Alice 2.2*. Cengage Learning. p. 122. ISBN 0-538-47866-7.
75. Md. Rezaul Karim; Sridhar Alla, (2017). *Scala and Spark for Big Data Analytics: Explore the concepts of functional programming, data streaming, and machine learning*. Packt Publishing Ltd. p. 87. ISBN 978-1-78355-050-0.
76. Lex Sheehan, (2017). *Learning Functional Programming in Go: Change the way you approach your applications using functional programming in Go*. Packt Publishing Ltd. p. 16. ISBN 978-1-78728-604-7.
77. Evelio Padilla, (2015). *Substation Automation Systems: Design and Implementation*. Wiley. p. 245. ISBN 978-1-118-98730-8.
78. "Multi-Paradigm Programming Language" (https://web.archive.org/web/20130821052407/https://developer.mozilla.org/en-US/docs/multiparadigmlanguage.html). *MDN Web Docs*. Mozilla Foundation. Archived from the original (https://developer.mozilla.org/en-US/docs/multiparadigmlanguage.html) on August 21, 2013.
79. Meyer, Bertrand (April 2009). "Viewpoint: Research evaluation for computer science" (https://pure.itu.dk/portal/da/publications/b474cea0-8288-11dd-b116-000ea68e967b). *Communications of the ACM*. **25** (4): 31–34. doi:10.1145/1498765.1498780 (https://doi.org/10.1145%2F1498765.1498780). S2CID 8625066 (https://api.semanticscholar.org/CorpusID:8625066).
80. Patterson, David (August 1999). "Evaluating Computer Scientists and Engineers For Promotion and Tenure" (http://cra.org/resources/bp-view/evaluating_computer_scientists_and_engineers_for_promotion_and_tenure/). Computing Research Association. Archived (https://web.archive.org/web/20150722020941/http://cra.org/resources/bp-view/evaluating_computer_scientists_and_engineers_for_promotion_and_tenure/) from the original on July 22, 2015. Retrieved July 19, 2015.
81. Fortnow, Lance (August 2009). "Viewpoint: Time for Computer Science to Grow Up" (https://doi.org/10.1145%2F1536616.1536631). *Communications of the ACM*. **52** (8): 33–35. doi:10.1145/1536616.1536631 (https://doi.org/10.1145%2F1536616.1536631).

Further reading

- Tucker, Allen B. (2004). *Computer Science Handbook* (2nd ed.). Chapman and Hall/CRC. ISBN 978-1-58488-360-9.
- Ralston, Anthony; Reilly, Edwin D.; Hemmendinger, David (2000). *Encyclopedia of Computer Science* (http://portal.acm.org/ralston.cfm) (4th ed.). Grove's Dictionaries. ISBN 978-1-56159-248-7. Archived (https://web.archive.org/web/20200608005417/https://dl.acm.org/doi/book/10.5555/1074100) from the original on June 8, 2020. Retrieved February 6, 2011.
- Edwin D. Reilly (2003). *Milestones in Computer Science and Information Technology* (https://archive.org/details/milestonesincomp0000reil). Greenwood Publishing Group. ISBN 978-1-57356-521-9.
- Knuth, Donald E. (1996). *Selected Papers on Computer Science*. CSLI Publications, Cambridge University Press.
- Collier, Bruce (1990). *The little engine that could've: The calculating machines of Charles Babbage* (http://robroy.dyndns.info/collier/index.html). Garland Publishing Inc. ISBN 978-0-8240-0043-1. Archived (https://web.archive.org/web/20070120190231/http://robroy.dyndns.info/collier/index.html) from the original on January 20, 2007. Retrieved May 4, 2013.
- Cohen, Bernard (2000). *Howard Aiken, Portrait of a computer pioneer*. The MIT press. ISBN 978-0-262-53179-5.
- Tedre, Matti (2014). *The Science of Computing: Shaping a Discipline*. CRC Press, Taylor & Francis.
- Randell, Brian (1973). *The origins of Digital computers, Selected Papers*. Springer-Verlag. ISBN 978-3-540-06169-4.
- Randell, Brian (October–December 1982). "From Analytical Engine to Electronic Digital Computer: The Contributions of Ludgate, Torres, and Bush" (https://web.archive.org/web/20130921055055/http://www.cs.ncl.ac.uk/publications/articles/papers/398.pdf) (PDF). *IEEE Annals of the History of Computing*. **4** (4): 327–341. Bibcode:1982IAHC....4d.327R (https://ui.adsabs.harvard.edu/abs/1982IAHC....4d.327R). doi:10.1109/mahc.1982.10042 (https://doi.org/10.1109%2Fmahc.1982.10042). S2CID 1737953 (https://api.semanticscholar.org/CorpusID:173795 3). Archived from the original (http://www.cs.ncl.ac.uk/research/pubs/articles/papers/398.pdf) (PDF) on September 21, 2013.
- Peter J. Denning. *Is computer science science? (http://portal.acm.org/citation.cfm?id=1053309&coll=&dl=ACM&CFID=15151515&CFTOKEN=61844618)*, Communications of the ACM, April 2005.
- Peter J. Denning, *Great principles in computing curricula (http://portal.acm.org/citation.cfm?id=971303&dl=ACM&coll=&CFID=15151515&CFTOKEN=6184618)*, Technical Symposium on Computer Science Education, 2004.

External links

- DBLP Computer Science Bibliography (http://dblp.uni-trier.de/)
- Association for Computing Machinery (http://www.acm.org/)
- Institute of Electrical and Electronics Engineers (https://www.ieee.org/)